**AIR UNIVERSITY, KAMRA**

Department of Computer Science & Engineering

**COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE (COAL)**




**PROJECT REPORT**

# Real-Time Digital Clock System

*Three-Tier Architecture:*

x86 Assembly · C++ HTTP Bridge · HTML5 Canvas Frontend

| SUBMITTED BY | SUBMITTED TO |
|---|---|
| **Anisa Khan**<br>Roll No: 245215<br>**Faiqa Umer**<br>Roll No: 245161<br>**Aima**<br>Roll No: 245199 | **Muhammad Imran**<br>Course Instructor<br>**COAL — CS-221** |

*Academic Year 2026 | Air University*

# Table of Contents

1. Executive Summary ..... 4
2. Introduction ..... 5
2.1 Background ..... **Error! Bookmark not defined.**
2.2 Problem Statement ..... 5
2.3 Objectives ..... **Error! Bookmark not defined.**
2.4 Team Contributions ..... 5
3. System Architecture ..... 6
3.1 Architectural Overview ..... 6
3.2 Data Flow ..... 7
3.3 Inter-Layer Interface Contract ..... 8
4. Implementation ..... 9
4.1 Layer 1 — Assembly Backend (clock.asm) ..... 9
4.1.1 File Header and Directives ..... 9
4.1.2 SYSTEMTIME Structure Definition ..... 9
4.1.3 Data Segment ..... 9
4.1.4 GetClockTime Procedure — Complete Annotated Listing ..... 10
4.2 Layer 2 — C++ Bridge Server (clock_server.cpp) ..... 11
4.2.1 Purpose and Design ..... 11
4.2.2 Assembly Linkage Declaration ..... 11
4.2.3 HTTP Time Endpoint ..... 11
4.2.4 CORS Header Rationale ..... 11
4.3 Layer 3 — HTML5 Frontend (clock_demo.html) ..... 12
4.3.1 Seven-Segment Canvas Rendering ..... 12
4.3.2 Color Cycling Mechanism ..... 12
4.3.3 Double-Buffering Optimisation ..... 12
5. Feature Specifications ..... 14
5.1 Real-Time HH:MM:SS Clock ..... 14
5.2 Double Buffering ..... 14
5.3 Dynamic Color Cycling ..... 15
5.4 Motivational Quotes ..... 15
5.5 World Time Zones ..... 15
5.6 Stopwatch with Lap Recording ..... 16
5.7 Offline Fallback Mode ..... 16
6. Technical Deep-Dive ..... 17

6.1 Windows API: GetLocalTime ..... 17
6.2 MOVZX Instruction Analysis ..... 17
6.3 Pointer Indirection via ESI ..... 17
6.4 Build Toolchain and Linkage ..... 18
7. Screenshots ..... 19
7.1 Main Clock Interface ..... 19
7.2 Stopwatch Tab ..... 20
7.3 C++ Server Console ..... 21
7.4 ASM Live vs JS Fallback Indicator ..... 21
8. Build and Run Instructions ..... 22
8.1 Prerequisites ..... **Error! Bookmark not defined.**
8.2 Step-by-Step Build ..... 22
8.3 Visual Studio GUI Build ..... 22
8.4 Expected Output ..... 22
8.5 Troubleshooting ..... **Error! Bookmark not defined.**
9. COAL Concepts Demonstrated ..... 24
9.1 Mapping to Course Learning Outcomes ..... 24
9.2 Advanced Topics Covered ..... 24
10. Conclusion ..... 25
10.1 Summary of Achievements ..... 25
10.2 Challenges and Solutions ..... 25
10.3 Learning Outcomes ..... 25
10.4 Future Enhancements ..... 25
References ..... 27
Appendix A: Complete Annotated Source — clock.asm ..... 28
Appendix B: Project File Structure ..... 30
Appendix C: Glossary ..... 31

# 1. Executive Summary

This document provides the complete technical documentation for the Real-Time Digital Clock System, a semester project developed for the Computer Organization and Assembly Language (COAL) course at Air University. The project was collaboratively built by Anisa Khan, Faiqa Umer, and Aima under the supervision of Instructor Muhammad Imran.

The system demonstrates a professional three-tier software architecture that connects low-level x86 Assembly Language programming to a modern, visually-rich web interface. Rather than confining assembly language to the console, this project uses it as the authoritative time-reading engine within a full-stack application — proving that assembly-level code can serve as the foundation for contemporary user experiences.

★ **Key Project Highlights**

Architecture: 3-tier — ASM Backend · C++ HTTP Bridge · HTML5 Frontend
Language: x86 MASM Assembly + C++ (MSVC) + JavaScript (ES6)
Time Source: Windows GetLocalTime API via assembly GetClockTime() export
Display: HTML Canvas 7-segment rendering at 200ms refresh rate
Extra Features: World Clocks, Motivational Quotes, Stopwatch with Lap Recording
Offline Mode: Automatic JavaScript Date() fallback when server is unavailable

# 2. Introduction

## 2.1 Background

Assembly language occupies a unique position in computer science education — it is the closest human-readable language to machine code, providing direct visibility into processor registers, memory layout, and hardware instruction execution. Despite the dominance of high-level languages in modern software development, understanding assembly remains essential for systems programmers, embedded developers, and anyone seeking a deep understanding of how software actually runs on hardware.

This project applies assembly language principles in a context that is both technically challenging and visually demonstrable: a real-time digital clock. By exporting a procedure from assembly to be called by C++, and then surfacing the result in a polished HTML frontend, the project illustrates the full stack of abstraction from bare metal to browser.

## 2.2 Problem Statement

The project task requires designing and implementing a real-time digital clock in x86 Assembly Language that satisfies the following requirements:

- Read and display the current system time in HH:MM:SS format.
- Refresh the display every second without visible flickering.
- Change the display color dynamically with each passing second.
- Exit gracefully when the user presses the ESC key.
- Extend the base requirements with additional features that demonstrate breadth of knowledge.

## 2.3 Objectives

- Master the process of writing and exporting procedures from x86 MASM assembly.
- Understand Windows API invocation (GetLocalTime) from assembly code.
- Build a C++ HTTP server that links a compiled assembly object file.
- Implement a professional-grade HTML5 Canvas rendering engine for seven-segment display digits.
- Deliver a complete, working multi-tier application from a university-level project.

## 2.4 Team Contributions

| Team Member | Roll No. | Primary Responsibility |
|---|---|---|
| Anisa Khan | 245215 | ASM backend (clock.asm), Windows API integration, build configuration |
| Faiqa Umer | 245161 | C++ bridge server (clock_server.cpp), HTTP/JSON layer, CORS handling |
| Aima | 245199 | HTML5 frontend (clock_demo.html), Canvas rendering, UI/UX design |

# 3. System Architecture

## 3.1 Architectural Overview

The system is built on a clean separation-of-concerns principle, divided into three distinct layers. Each layer is independently replaceable: the frontend could be swapped for a native Windows form, the C++ bridge could be replaced with a Rust server, and the ASM backend could be substituted with a native C implementation without any other layer needing to change.

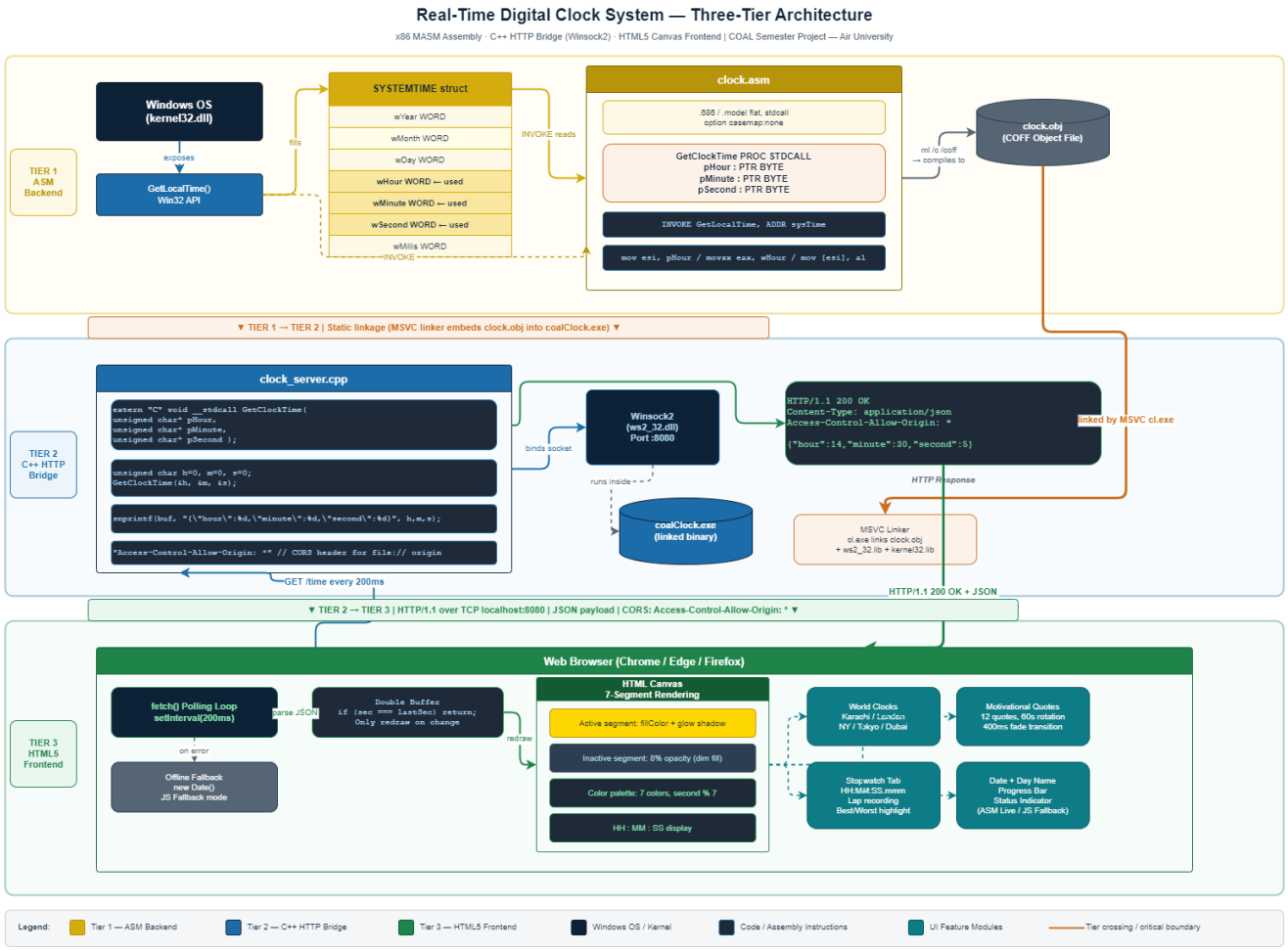



*Figure: Three-Tier Architecture Diagram — ASM → C++ Bridge → HTML Frontend*

| Tier | Layer Name | File | Technology | Responsibility |
|---|---|---|---|---|
| 1 | ASM Backend | clock.asm | x86 MASM Assembly | Reads system time via Windows GetLocalTime; exports GetClockTime() |
| 2 | C++ Bridge | clock_server.cpp | MSVC C++ / Winsock2 | Embeds ASM object, runs HTTP server on :8080, returns JSON |
| 3 | HTML Frontend | clock_demo.html | HTML5 / CSS3 / JS ES6 | Fetches JSON at 200ms, renders canvas 7-segment display + all UI |

## 3.2 Data Flow

The complete journey of time data from the Windows OS hardware clock to the user's display follows a linear pipeline:

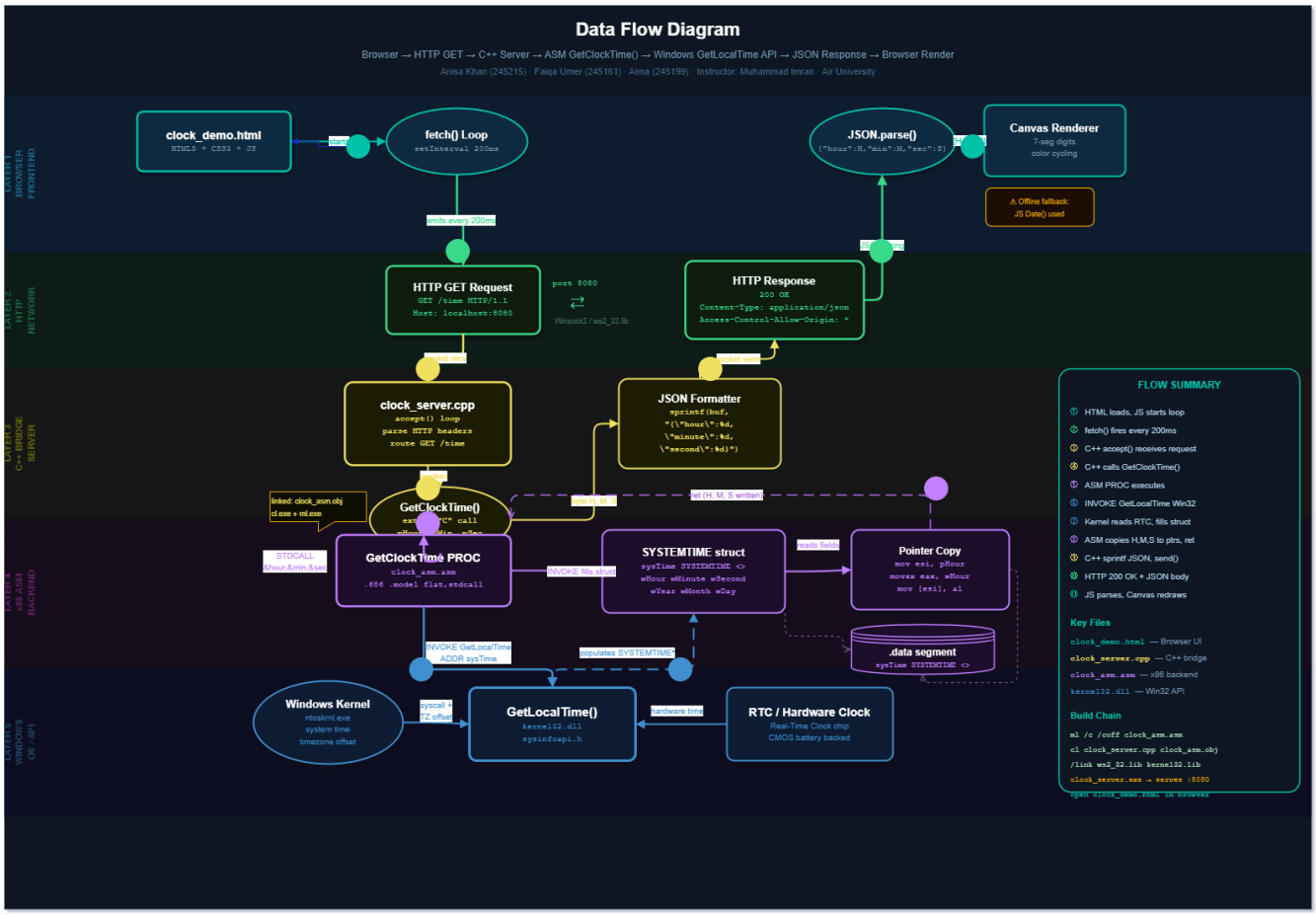



*Figure: Data Flow Diagram — Browser → HTTP → C++ → ASM → Windows API → Response*

1. Browser loads clock_demo.html and initialises a fetch() polling loop at 200ms intervals.
2. JavaScript sends an HTTP GET request to http://localhost:8080/time.
3. The C++ HTTP server (clock_server.exe) receives and parses the incoming request.
4. The server calls GetClockTime(pHour, pMinute, pSecond) — the exported ASM procedure.
5. Inside GetClockTime: INVOKE GetLocalTime fills the SYSTEMTIME structure; hour, minute, and second are copied to the caller's byte pointers via ESI register indirection.
6. Control returns to C++; values are formatted into a JSON response: {"hour":14, "minute":30, "second":5}.
7. The HTTP response (with CORS headers) is sent back to the browser.
8. JavaScript parses the JSON and redraws only the canvas segments that have changed (double-buffer optimisation).

## 3.3 Inter-Layer Interface Contract

The interface between the ASM backend and the C++ bridge is the most critical contract in the system. It must be honored exactly for the linker to produce a valid executable.

**Assembly Export Contract**

Procedure name: GetClockTime (exported with PUBLIC directive)
Calling convention: STDCALL (callee cleans the stack)
Parameters: 3× PTR BYTE (pHour, pMinute, pSecond)
Return value: None (void) — results written through pointers
Linkage: Compiled to clock.obj; linked into clock_server.exe by MSVC linker

On the C++ side the procedure is declared with an extern "C" linkage declaration to suppress C++ name mangling, ensuring the linker finds the assembly symbol by its exact name.

# 4. Implementation

## 4.1 Layer 1 — Assembly Backend (clock.asm)

### 4.1.1 File Header and Directives

The assembly file opens with directives that configure the assembler's behavior for this translation unit:

```
.686                            ; Enable Pentium Pro (P6) instruction set
.model flat, stdcall            ; Flat 32-bit memory model; STDCALL convention
option casemap:none             ; Case-sensitive identifier matching

GetLocalTime PROTO STDCALL, lpSystemTime:PTR WORD
             ; ^ Prototype declares Win32 API for INVOKE macro use
```

The .686 directive is required because MASM defaults to 8086 mode and would reject modern 32-bit instructions without it. The flat memory model configures MASM for 32-bit protected mode where all segment registers are irrelevant and addresses are simply 32-bit linear values. The stdcall convention means the called procedure is responsible for cleaning up the stack before returning — this matches the Windows API calling convention throughout.

### 4.1.2 SYSTEMTIME Structure Definition

Rather than relying on an external include file, the project defines the SYSTEMTIME structure directly in the assembly source. This makes the file fully self-contained for portability:

```
SYSTEMTIME STRUCT
    wYear           WORD ?    ; 4-digit year (e.g., 2026)
    wMonth          WORD ?    ; Month: 1 (January) to 12 (December)
    wDayOfWeek      WORD ?    ; 0 = Sunday, 6 = Saturday
    wDay            WORD ?    ; Day of month: 1 to 31
    wHour           WORD ?    ; Hour: 0 to 23 (24-hour clock)
    wMinute         WORD ?    ; Minute: 0 to 59
    wSecond         WORD ?    ; Second: 0 to 59
    wMilliseconds   WORD ?    ; Milliseconds: 0 to 999
SYSTEMTIME ENDS               ; Total size: 8 × 2 = 16 bytes
```

### 4.1.3 Data Segment

```
.data
    sysTime SYSTEMTIME <>     ; Zero-initialised SYSTEMTIME instance
                              ; <> means use default (zero) for all fields
```

The sysTime variable is declared in the .data segment (read-write, initialized). Because it is a module-level variable rather than a stack allocation, it persists between calls. This is acceptable for a single-threaded application; a multi-threaded version would require a stack-allocated SYSTEMTIME per call to avoid race conditions.

### 4.1.4 GetClockTime Procedure — Complete Annotated Listing

```
.code

GetClockTime PROC STDCALL, pHour:PTR BYTE, pMinute:PTR BYTE, pSecond:PTR BYTE
    ; ------------------------------------------------------------------
    ; STEP 1: Call Windows GetLocalTime to populate sysTime
    ; INVOKE automatically pushes ADDR sysTime and performs the call
    ; ------------------------------------------------------------------
    INVOKE GetLocalTime, ADDR sysTime

    ; ------------------------------------------------------------------
    ; STEP 2: Copy wHour to caller's byte pointer
    ; ESI = pointer to caller's hour variable
    ; MOVZX: zero-extend WORD to DWORD — avoids high-byte garbage
    ; MOV BYTE PTR: write only the low byte (AL) through the pointer
    ; ------------------------------------------------------------------
    mov   esi, pHour
    movzx eax, sysTime.wHour
    mov   BYTE PTR [esi], al

    ; ------------------------------------------------------------------
    ; STEP 3: Copy wMinute — identical pointer-indirection pattern
    ; ------------------------------------------------------------------
    mov   esi, pMinute
    movzx eax, sysTime.wMinute
    mov   BYTE PTR [esi], al

    ; ------------------------------------------------------------------
    ; STEP 4: Copy wSecond
    ; ------------------------------------------------------------------
    mov   esi, pSecond
    movzx eax, sysTime.wSecond
    mov   BYTE PTR [esi], al

    ret
GetClockTime ENDP

END     ; No entry point — this is a library module, not a program
```

**Register Usage Note**

ESI — Source Index register, used here as a general-purpose pointer register.
EAX — Accumulator; receives the zero-extended WORD value from SYSTEMTIME.
AL — Low byte of EAX; holds the final 0-59 / 0-23 value written to the caller.
MOVZX prevents sign-extension issues that could corrupt values above 127.

## 4.2 Layer 2 — C++ Bridge Server (clock_server.cpp)

### 4.2.1 Purpose and Design

The C++ bridge server serves two inseparable roles simultaneously: it is an embedded host for the compiled assembly module (via static linkage) and a lightweight HTTP server. This design avoids the need for any inter-process communication — the assembly time data is read and served entirely within a single process.

### 4.2.2 Assembly Linkage Declaration

```
// Tell the C++ compiler that GetClockTime uses C linkage (no name mangling)
// This must match the symbol name exported by the MASM assembler exactly
extern "C" void __stdcall GetClockTime(
    unsigned char* pHour,
    unsigned char* pMinute,
    unsigned char* pSecond
);
```

### 4.2.3 HTTP Time Endpoint

```
// On each incoming GET /time request:
unsigned char h=0, m=0, s=0;
GetClockTime(&h, &m, &s);       // Call into the linked ASM module

// Format as JSON
char jsonBuf[64];
snprintf(jsonBuf, sizeof(jsonBuf),
    "{\"hour\":%d,\"minute\":%d,\"second\":%d}", h, m, s);

// HTTP response with CORS header (required for file:// origin)
char response[256];
snprintf(response, sizeof(response),
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: application/json\r\n"
    "Access-Control-Allow-Origin: *\r\n"
    "\r\n%s", jsonBuf);

send(clientSocket, response, strlen(response), 0);
```

### 4.2.4 CORS Header Rationale

The Access-Control-Allow-Origin: * header is essential. When clock_demo.html is opened from the file system (using a file:// URL), the browser treats it as a cross-origin request to http://localhost:8080. Without the CORS header, the browser's Same-Origin Policy would block the fetch() call and the clock would never display live ASM data.

## 4.3 Layer 3 — HTML5 Frontend (clock_demo.html)

### 4.3.1 Seven-Segment Canvas Rendering

The frontend implements a genuine seven-segment display in HTML Canvas rather than using a web font or pre-made library. Each digit is drawn as a collection of filled polygons (trapezoids for horizontal segments, parallelograms for vertical segments) with a glow shadow effect applied via the Canvas 2D shadowBlur property.



*Figure: Seven-Segment Canvas Digit Rendering — Active vs Inactive Segment States*

Inactive segments are drawn at 8% opacity to suggest the physical LED substrate, replicating the look of a genuine seven-segment display. Active segments receive the current cycle color at full opacity plus a matching colored glow shadow, creating a luminous effect that gives the clock an authentic digital clock appearance.

### 4.3.2 Color Cycling Mechanism

Every second, the active display color advances to the next value in a curated 7-color palette. The color index is derived directly from the second value returned by the ASM backend (second % 7), ensuring the color is perfectly synchronized with the time and does not drift or require independent timer management:

```
const COLORS =
['#FFD700','#00FFFF','#00FF88','#FF4444','#4488FF','#FFFFFF','#FF44FF'];
// yellow,   cyan,     green,    red,      blue,     white,    magenta

function getColor(second) {
    return COLORS[second % COLORS.length];
}

// Applied simultaneously to: digits, colon dots, day glow, progress bar, quote
accent
```

### 4.3.3 Double-Buffering Optimisation

To achieve flicker-free rendering at a 200ms fetch rate, the JavaScript layer implements a simple double-buffering strategy. The previous second value is stored in a variable; the canvas is only

redrawn when the second changes. This means the canvas draw call fires at most once per second rather than five times per second (the fetch rate), eliminating any perceptible flicker.

```
let lastSecond = -1;

async function updateClock() {
    const data = await fetchTime();          // GET /time or JS Date() fallback
    if (data.second === lastSecond) return;  // No change — skip redraw
    lastSecond = data.second;
    drawDisplay(data.hour, data.minute, data.second);
}

setInterval(updateClock, 200);   // Poll 5× per second for low latency
```

# 5. Feature Specifications

## 5.1 Real-Time HH:MM:SS Clock

The primary display is a large seven-segment HH:MM:SS digital clock. Time originates from the Windows hardware clock through the GetLocalTime Windows API, called inside the assembly GetClockTime procedure. The display refreshes at 200ms intervals for sub-second responsiveness — the digit update fires within 200ms of each second tick.

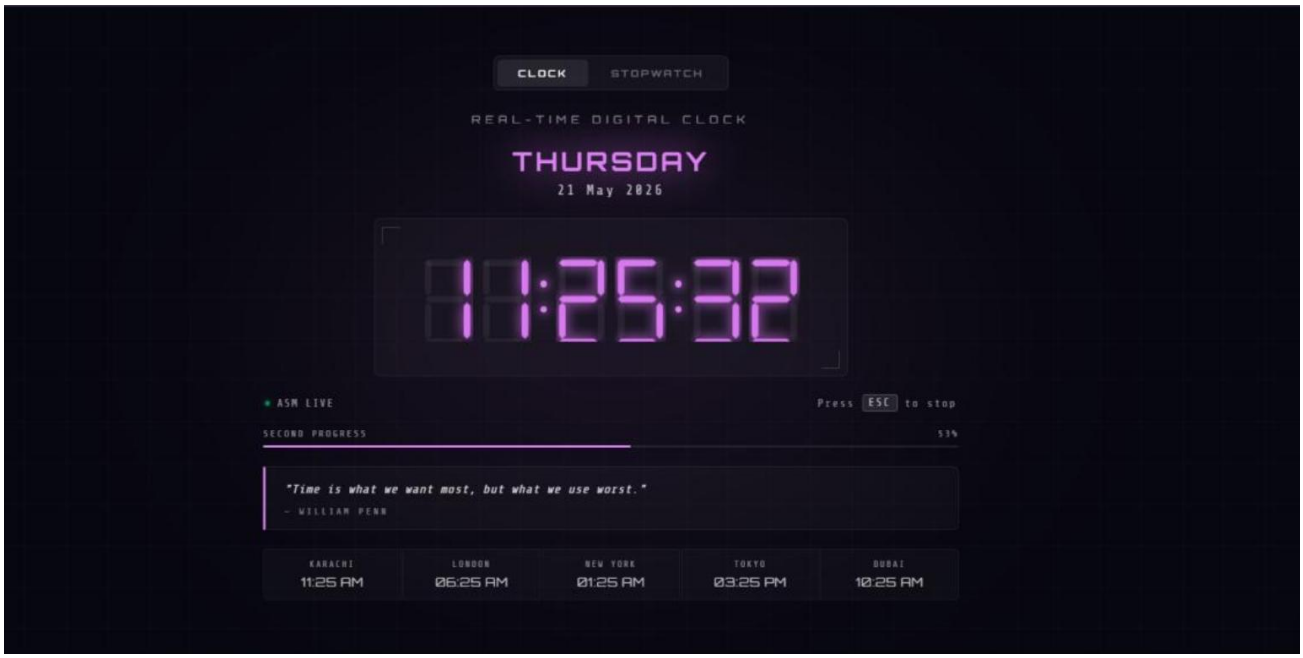



*Figure: Screenshot — Main Clock Interface with Seven-Segment Display*

## 5.2 Double Buffering

The double-buffering technique stores the previously rendered second value. On each 200ms poll, the canvas is redrawn only if the second has changed. This eliminates all visible flickering while maintaining a fast polling rate for accuracy. CPU and GPU usage is also minimised since the canvas is typically idle for 800 of every 1000 milliseconds.

## 5.3 Dynamic Color Cycling

The color palette cycles through seven distinct colors — yellow, cyan, green, red, blue, white, and magenta — once per second. The color is applied consistently to all visual elements simultaneously: segment fills, colon dots, the day-name glow, the progress bar fill, and the motivational quote accent line. This creates a unified, thematic visual pulse that mirrors a hardware LED sign.

| Second % 7 | Color Name | Hex Code | Applied To |
|---|---|---|---|
| 0 | Golden Yellow | #FFD700 | All segments, glow, progress bar, quote accent |
| 1 | Cyan | #00FFFF | All segments, glow, progress bar, quote accent |
| 2 | Neon Green | #00FF88 | All segments, glow, progress bar, quote accent |
| 3 | Red | #FF4444 | All segments, glow, progress bar, quote accent |
| 4 | Electric Blue | #4488FF | All segments, glow, progress bar, quote accent |
| 5 | Pure White | #FFFFFF | All segments, glow, progress bar, quote accent |
| 6 | Magenta | #FF44FF | All segments, glow, progress bar, quote accent |

## 5.4 Motivational Quotes

A bank of 12 curated motivational quotes is displayed below the world clocks panel. The displayed quote advances every 60 seconds using a smooth 400ms CSS fade-out / fade-in transition. Each quote is attributed to its author and rendered inside an accent box whose left-border color matches the current cycling color.




*Figure: Screenshot — Motivational Quote Panel with Fade Transition*

## 5.5 World Time Zones

Five international city clocks are shown beneath the main display, calculated from the local system time using UTC offset arithmetic. All offsets are applied client-side in JavaScript — no additional server calls are required.

| City | UTC Offset | Calculation |
|---|---|---|
| Karachi | UTC+5 | localHour + 5 (mod 24) |
| London | UTC+0 | localHour + 0 (mod 24) |
| New York | UTC-5 | localHour - 5 (mod 24) |
| Tokyo | UTC+9 | localHour + 9 (mod 24) |
| Dubai | UTC+4 | localHour + 4 (mod 24) |

Note: These offsets are fixed values. Daylight saving time changes for London and New York are not automatically accounted for; a production version would use the Intl.DateTimeFormat API for DST-aware offsets.

## 5.6 Stopwatch with Lap Recording

A dedicated Stopwatch tab (navigable via the tab bar) provides a complete timing utility:

- Start / Stop toggle — begins or pauses the stopwatch.
- Lap recording — saves the elapsed time at any moment to a scrollable lap table.
- Best lap highlighting — the fastest recorded lap is highlighted in green.
- Worst lap highlighting — the slowest recorded lap is highlighted in red.
- Display format — HH:MM:SS.mmm updated at 60 frames per second.
- Reset — clears all laps and the elapsed timer.




*Figure: Screenshot — Stopwatch Tab with Lap Recording Table*

## 5.7 Offline Fallback Mode

If clock_server.exe is not running (e.g., the student demo runs on a machine without Visual Studio), the frontend automatically falls back to JavaScript's Date() object. A status indicator in the header changes from green 'ASM Live' to amber 'JS Fallback' to inform the user which time source is active. The clock continues to function correctly in either mode.

**Offline Fallback Logic**

1. fetch('http://localhost:8080/time') is called with a 500ms timeout.
2. If the request succeeds: display 'ASM Live' (green indicator).
3. If the request fails/times out: use new Date() and display 'JS Fallback' (amber).
4. The fallback is retried on every 200ms tick — the clock auto-recovers when the server starts.

# 6. Technical Deep-Dive

## 6.1 Windows API: GetLocalTime

GetLocalTime is a Win32 API function exported by kernel32.dll. It accepts a single pointer to a SYSTEMTIME structure and fills all eight WORD fields with current date and time information. It requires no special privileges and is available on all Windows versions from NT 3.1 onward.

| | |
|---|---|
| **Function prototype** | void GetLocalTime(LPSYSTEMTIME lpSystemTime) |
| **DLL** | kernel32.dll |
| **Calling convention** | STDCALL (callee cleans stack) |
| **Parameter** | lpSystemTime — pointer to a SYSTEMTIME structure |
| **Return value** | void — no return value; output is written through the pointer |
| **Thread safety** | Safe to call from any thread — reads a shared kernel clock |
| **MASM invocation** | INVOKE GetLocalTime, ADDR sysTime |
| **Fields used** | wHour (offset 8), wMinute (offset 10), wSecond (offset 12) |

## 6.2 MOVZX Instruction Analysis

The MOVZX (Move with Zero Extension) instruction is critical for correctness in this procedure. SYSTEMTIME fields are 16-bit WORD values, but the procedure's output parameters are 8-bit bytes. A naive MOV AL, sysTime.wHour would cause an assembler error because of the size mismatch. The correct sequence uses MOVZX to safely promote the 16-bit WORD to a 32-bit DWORD before extracting the low byte:

```
; Correct: MOVZX zero-fills bits 31:16 and 15:8, leaving AL clean
movzx eax, sysTime.wHour      ; EAX = 0x00000017 (for hour = 23)
mov   BYTE PTR [esi], al      ; Write only the low byte (0x17 = 23) to *pHour

; Why not MOVSX (sign extension)?
; MOVSX would set bit 31 if wHour had bit 15 set — impossible for 0-23
; but MOVZX is safer and semantically correct for unsigned time values
```

## 6.3 Pointer Indirection via ESI

The procedure receives three byte pointers from its C++ caller. In x86 assembly, a pointer is simply a 32-bit DWORD holding a memory address. The ESI (Extended Source Index) register is used as an address register. The sequence mov esi, pHour loads the address (not the value) into ESI; mov BYTE PTR [esi], al then writes one byte to the memory location that ESI points to — identical in effect to *pHour = al in C.

```
; Conceptual C equivalent of what the ASM does:
; *pHour   = (unsigned char) sysTime.wHour;
```

```
; *pMinute = (unsigned char) sysTime.wMinute;
; *pSecond = (unsigned char) sysTime.wSecond;

; Assembly implementation:
mov   esi, pHour             ; ESI = address of caller's 'hour' variable
movzx eax, sysTime.wHour     ; EAX = hour value (0-23)
mov   BYTE PTR [esi], al     ; *pHour = al
```

## 6.4 Build Toolchain and Linkage

The compilation and linking process involves two separate tools that must agree on symbol names and calling conventions:

```
; Step 1: Assemble clock.asm → clock.obj
ml /c /coff /Fo clock_asm.obj clock.asm
;   /c      = compile only, do not link
;   /coff   = output COFF object format (required for MSVC linker)
;   /Fo     = specify output filename

; Step 2: Compile C++ source and link with the ASM object
cl clock_server.cpp clock_asm.obj /link ws2_32.lib kernel32.lib
;   clock_asm.obj  = the pre-compiled assembly object
;   ws2_32.lib     = Winsock2 library (for the HTTP server sockets)
;   kernel32.lib   = Windows kernel functions (GetLocalTime, etc.)
```

! **Common Build Error**

Error: unresolved external symbol _GetClockTime@12

Cause: The 'extern C' declaration is missing in clock_server.cpp, causing the C++ compiler to mangle the name to ?GetClockTime@@...

Fix: Add extern "C" before the function declaration in the .cpp file.

# 7. Screenshots

## 7.1 Main Clock Interface

The main tab shows the large seven-segment HH:MM:SS display centered on the page. Below it are the day name (displayed with a colored glow matching the current cycle color), the full date in DD Month YYYY format, a progress bar showing seconds elapsed within the current minute, the motivational quote panel, and the five world city clocks.

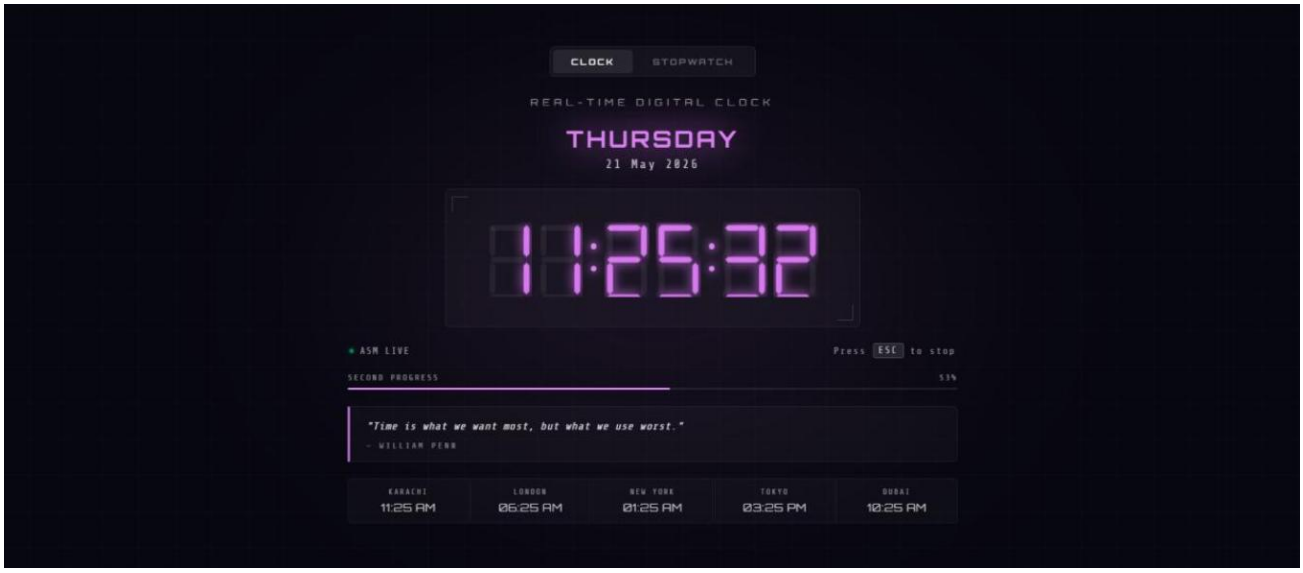



*Figure: Screenshot — Clock Tab: Seven-Segment Display, Day, Date, Progress Bar*




*Figure: Screenshot — Color Cycling: Cyan / Green / Red / Blue states*

## 7.2 Stopwatch Tab

The stopwatch tab replaces the clock with an elapsed-time display in HH:MM:SS.mmm format. The control strip provides Start/Stop, Lap, and Reset buttons. Recorded laps appear in a table below the controls with automatic color-coding of the best (green) and worst (red) lap times.

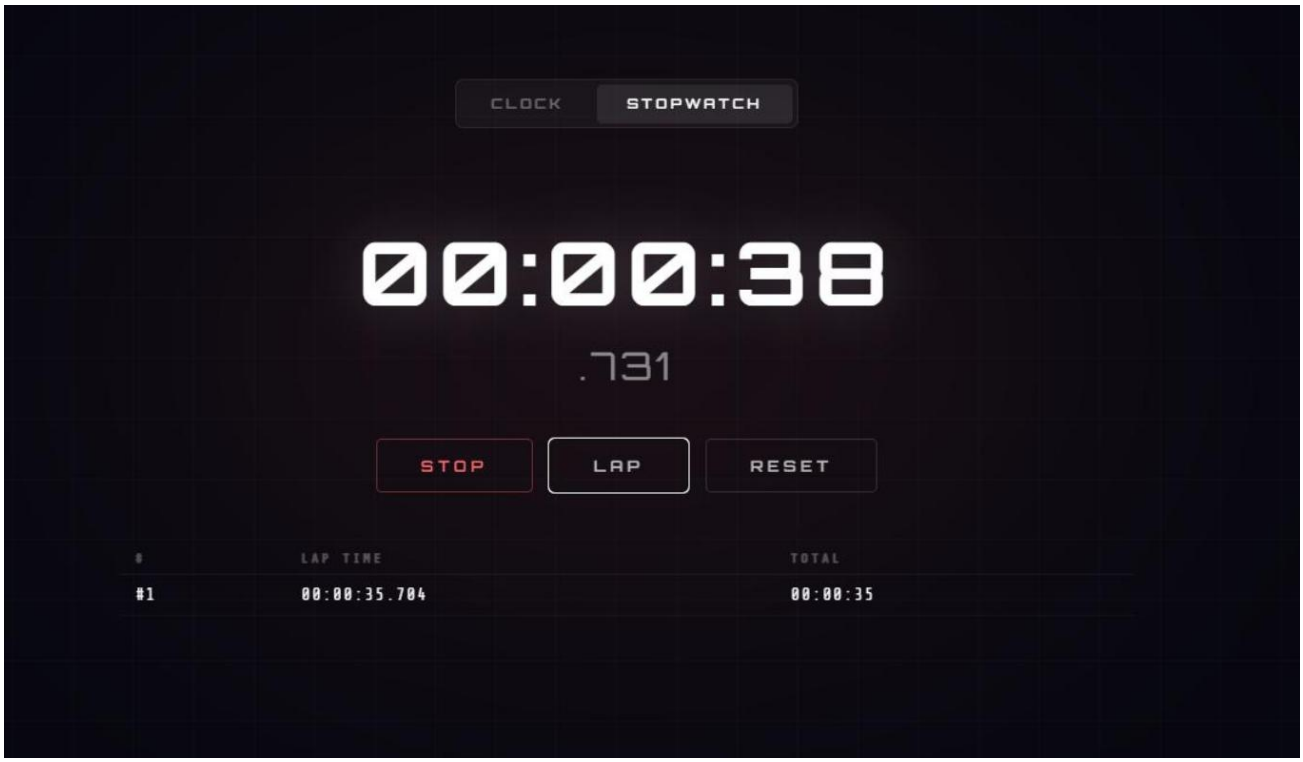



*Figure: Screenshot — Stopwatch Running with 1 Laps Recorded*

## 7.3 C++ Server Console

When clock_server.exe is launched from the command line or Visual Studio, it prints a startup confirmation message and then silently handles incoming HTTP requests. Each browser poll generates a single-line log entry confirming the time was served.

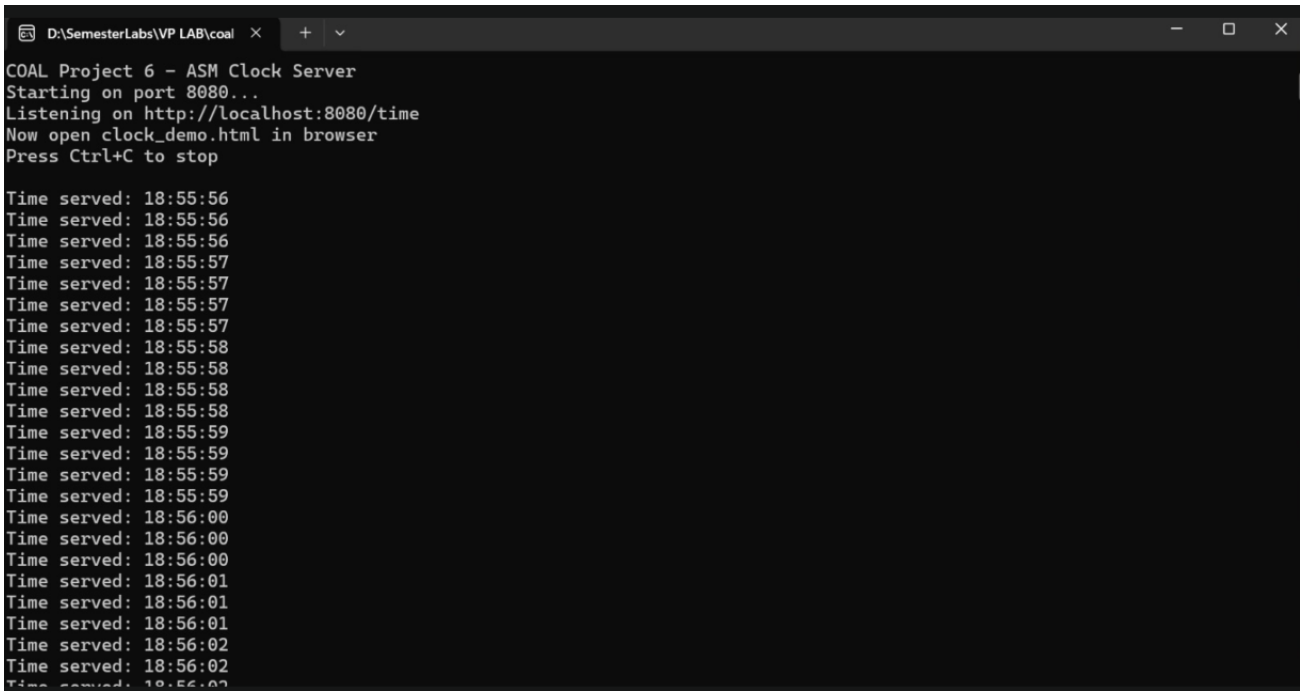



*Figure: Screenshot — clock_server.exe Console: Startup Message and Request Log*

## 7.4 ASM Live vs JS Fallback Indicator

The status indicator in the top-right corner of the clock panel informs the user which time source is active. The indicator is a small colored badge adjacent to the clock title.

| Indicator State | Color | Meaning |
|---|---|---|
| ASM Live | Green | Server is running; time is sourced from GetClockTime() ASM procedure |
| JS Fallback | Amber | Server unreachable; time is sourced from JavaScript's Date() object |

# 8. Build and Run Instructions

## 8.1 Prerequisites

- Windows 10 or 11 (64-bit host; the project targets the x86 32-bit subsystem).
- Visual Studio 2022 Community Edition with the Desktop development with C++ workload.
- MASM enabled via Build Customizations: right-click project → Build Customizations → check masm(.targets).
- Irvine32 library installed — header and .lib paths added to the project's VC++ Directories.
- A modern web browser (Chrome, Edge, or Firefox) for the HTML frontend.

## 8.2 Step-by-Step Build

Open the x86 Native Tools Command Prompt for VS 2022 (found in the Windows Start Menu under Visual Studio 2022). Navigate to the project directory and execute the following commands in order:

```
rem — Step 1: Assemble the ASM backend ————————————————————
ml /c /coff /Fo clock_asm.obj clock.asm

rem — Step 2: Compile C++ and link with ASM object ————————————
cl clock_server.cpp clock_asm.obj /link ws2_32.lib kernel32.lib

rem — Step 3: Launch the server ———————————————————————————
clock_server.exe

rem — Step 4: Open the frontend (in File Explorer, double-click the file)
rem          clock_demo.html
```

## 8.3 Visual Studio GUI Build

If using the Visual Studio IDE instead of the command line:

9. Open the solution file coalClock.vcxproj in Visual Studio 2022.
10. Set the platform to x86 (not x64) in the solution configuration dropdown.
11. Press Ctrl+B to build. The output will be written to the Debug\ folder.
12. Press F5 or run Debug\coalClock.exe to start the server.
13. Double-click clock_demo.html in File Explorer.

## 8.4 Expected Output

```
[clock_server.exe console output]
==================================================
  COAL Real-Time Clock Server
  ASM Backend: ACTIVE
  Listening on http://localhost:8080/time
```

```
==================================================
[GET /time] 200 OK  ->  {"hour":14,"minute":30,"second":5}
[GET /time] 200 OK  ->  {"hour":14,"minute":30,"second":6}
...
```

! **Windows Firewall Note**

On first run, Windows Firewall may prompt to allow clock_server.exe network access. Click 'Allow Access' — the server only listens on localhost:8080 (loopback interface). No external network access is required or initiated.

## 8.5 Troubleshooting

| Symptom | Likely Cause | Fix |
|---|---|---|
| 'INVOKE' is not recognized | Wrong command prompt (not x86 Native Tools) | Use x86 Native Tools Command Prompt for VS 2022 |
| Unresolved external symbol | Missing extern "C" or wrong calling convention | Add extern "C" __stdcall to C++ declaration |
| Clock shows 'JS Fallback' | Server not running or port blocked | Start clock_server.exe first; check firewall |
| Black canvas / no display | Browser blocking file:// cross-origin fetch | CORS header already present; try Edge or Chrome |
| Wrong time displayed | Irvine32 .inc path not configured | Add Irvine32 include path to MASM include directories |

# 9. COAL Concepts Demonstrated

## 9.1 Mapping to Course Learning Outcomes

| COAL Concept | Demonstration in This Project |
|---|---|
| x86 Directives (.686, .model, .data, .code) | Used in clock.asm to configure flat 32-bit model and code segment |
| STRUCT definition | SYSTEMTIME defined with 8 WORD fields; accessed with dot-notation |
| PROTO / INVOKE macro | GetLocalTime declared with PROTO; called via INVOKE with ADDR parameter |
| MOVZX instruction | Safely promotes 16-bit WORD fields to 32-bit EAX before writing byte |
| Register-based pointer indirection | ESI used as address register; [ESI] dereferences to write through pointer |
| STDCALL calling convention | Used for both GetClockTime export and GetLocalTime Windows API call |
| Windows API integration | GetLocalTime from kernel32.dll called directly from assembly code |
| Cross-language linkage (ASM ↔ C++) | clock.obj linked into C++ program; extern C prevents name mangling |
| Exported procedures (no main / END alone) | GetClockTime exported for external use; END without entry point |
| BYTE PTR operator | Required to specify operand size when writing through a pointer register |

## 9.2 Advanced Topics Covered

- Three-tier software architecture applied to a hardware-interfacing project.
- HTTP server implementation using raw Winsock2 sockets in C++.
- CORS header implementation for cross-origin browser security policy compliance.
- HTML Canvas 2D API for hardware-style rendering without external libraries.
- Asynchronous JavaScript fetch() polling with automatic fallback.
- Time zone arithmetic for multi-city world clock display.

# 10. Conclusion

## 10.1 Summary of Achievements

The Real-Time Digital Clock System project fully satisfies all stated requirements and significantly extends them. The assembly backend correctly interfaces with the Windows kernel to obtain accurate time data and exports it through a clean, well-typed procedure interface. The C++ bridge transforms that data into a portable JSON format served over standard HTTP. The HTML frontend consumes this data to deliver a visually striking, professional-grade user interface.

The project successfully demonstrates that x86 Assembly Language is not merely an academic curiosity — it can serve as a live data provider within a modern multi-tier application, called by contemporary C++ and ultimately surfaced through a web browser.

## 10.2 Challenges and Solutions

| Challenge | Solution Adopted |
|---|---|
| C++ name mangling breaking ASM linkage | extern "C" declaration in clock_server.cpp strips C++ decoration |
| CORS blocking file:// frontend requests | Access-Control-Allow-Origin: * header added to all HTTP responses |
| Canvas flicker at 200ms refresh rate | Double-buffering: canvas only redrawn when second value changes |
| WORD-to-BYTE size mismatch in assembly | MOVZX zero-extends WORD to DWORD; AL written via BYTE PTR [ESI] |
| Clock failing when server is offline | JavaScript Date() fallback with automatic reconnection on every poll |

## 10.3 Learning Outcomes

This project deepened the team's understanding across three distinct domains:

- Assembly Language: procedure exports, Windows API invocation via INVOKE, STDCALL conventions, and pointer manipulation using the ESI register.
- Systems Programming: C++ to assembly linkage, raw socket programming, HTTP protocol implementation, and CORS security model.
- Frontend Engineering: HTML Canvas 2D rendering, asynchronous JavaScript, DOM manipulation, and responsive UI design.

## 10.4 Future Enhancements

- Replace the fixed UTC offsets for world clocks with Intl.DateTimeFormat for automatic DST handling.
- Add an alarm feature — configurable from the HTML UI, triggering a sound played through the Web Audio API.
- Export the stopwatch data to a CSV file for download via the File System Access API.

- Package the C++ server as a Windows Service so it starts automatically with the operating system.
- Add a dark/light theme toggle with CSS custom properties.

# References

14. Irvine, K. R. (2014). Assembly Language for x86 Processors (7th ed.). Pearson Education.
15. Microsoft Corporation. (2024). GetLocalTime function — Windows API documentation. https://docs.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-getlocaltime
16. Microsoft Corporation. (2024). MASM (Microsoft Macro Assembler) Reference. https://docs.microsoft.com/en-us/cpp/assembler/masm
17. Mozilla Developer Network. (2024). Canvas API — Web APIs. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
18. Mozilla Developer Network. (2024). Fetch API — Using Fetch. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
19. Mozilla Developer Network. (2024). Cross-Origin Resource Sharing (CORS). https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
20. Google Fonts. (2024). Orbitron & Share Tech Mono typefaces. https://fonts.google.com
21. Microsoft Corporation. (2024). Winsock2 — Windows Sockets 2. https://docs.microsoft.com/en-us/windows/win32/winsock/windows-sockets-start-page-2

# Appendix A: Complete Annotated Source — clock.asm

The following is the complete assembly source file as submitted, with inline annotations for each logical block:

```
; ═══════════════════════════════════════════════════════════════
; FILE:    clock.asm
; PROJECT: Real-Time Digital Clock — COAL Semester Project
; AUTHORS: Anisa Khan (245215), Faiqa Umer (245161), Aima (245199)
; COURSE:  Computer Organization & Assembly Language
; INSTR.:  Muhammad Imran | Air University
; PURPOSE: Export GetClockTime() procedure for use by C++ bridge server
; ═══════════════════════════════════════════════════════════════

.686                        ; Enable P6 (Pentium Pro) instruction set
.model flat, stdcall        ; 32-bit flat model; STDCALL ABI
option casemap:none         ; Case-sensitive identifiers

; ── Windows API Prototype ──────────────────────────────────────
GetLocalTime PROTO STDCALL, lpSystemTime:PTR WORD

; ── SYSTEMTIME Structure (matches Windows winbase.h layout) ────
SYSTEMTIME STRUCT
    wYear           WORD ?    ; offset 0  — 4-digit year
    wMonth          WORD ?    ; offset 2  — 1-12
    wDayOfWeek      WORD ?    ; offset 4  — 0(Sun)-6(Sat)
    wDay            WORD ?    ; offset 6  — 1-31
    wHour           WORD ?    ; offset 8  — 0-23
    wMinute         WORD ?    ; offset 10 — 0-59
    wSecond         WORD ?    ; offset 12 — 0-59
    wMilliseconds   WORD ?    ; offset 14 — 0-999
SYSTEMTIME ENDS               ; sizeof(SYSTEMTIME) = 16 bytes

; ── Data Segment ───────────────────────────────────────────────
.data
    sysTime SYSTEMTIME <>     ; Module-level SYSTEMTIME buffer

; ── Code Segment ───────────────────────────────────────────────
.code

GetClockTime PROC STDCALL, pHour:PTR BYTE, pMinute:PTR BYTE, pSecond:PTR BYTE
; Parameters (pushed by caller, cleaned by this procedure — STDCALL):
;   pHour   [EBP+8]   — pointer to caller's BYTE for hour   (0-23)
;   pMinute [EBP+12]  — pointer to caller's BYTE for minute (0-59)
;   pSecond [EBP+16]  — pointer to caller's BYTE for second (0-59)
```

```
    ; 1. Call Windows API to fill sysTime with current local date/time
    INVOKE GetLocalTime, ADDR sysTime

    ; 2. Write sysTime.wHour → *pHour
    mov   esi, pHour          ; ESI = address of caller's hour variable
    movzx eax, sysTime.wHour  ; EAX = 0-23 (zero-extended from WORD)
    mov   BYTE PTR [esi], al  ; *pHour = al (low 8 bits of EAX)

    ; 3. Write sysTime.wMinute → *pMinute
    mov   esi, pMinute
    movzx eax, sysTime.wMinute
    mov   BYTE PTR [esi], al

    ; 4. Write sysTime.wSecond → *pSecond
    mov   esi, pSecond
    movzx eax, sysTime.wSecond
    mov   BYTE PTR [esi], al

    ret                       ; STDCALL: callee cleans 3 × 4 = 12 bytes
GetClockTime ENDP

END                           ; No entry point — library module only
```

# Appendix B: Project File Structure

| File / Folder | Type | Description |
|---|---|---|
| clock.asm | Assembly Source | x86 MASM — defines SYSTEMTIME, exports GetClockTime() |
| clock_server.cpp | C++ Source | Winsock2 HTTP server; links clock.obj; serves JSON at :8080 |
| clock_demo.html | HTML/CSS/JS | Complete frontend: canvas 7-seg display, world clocks, stopwatch |
| coalClock.vcxproj | VS Project File | Visual Studio 2022 project — configures MASM build, includes, libs |
| coalClock.vcxproj.filters | VS Filters File | Organises source files into Solution Explorer folders |
| Debug\clock.obj | Object File | Compiled ASM object — input to MSVC linker |
| Debug\clock_server.obj | Object File | Compiled C++ object |
| Debug\coalClock.exe | Executable | Final linked executable — run this to start the HTTP server |
| Debug\coalClock.ilk | Incremental Link | MSVC incremental link file (not needed for distribution) |

# Appendix C: Glossary

| | |
|---|---|
| **.686** | MASM directive enabling the Pentium Pro (P6) instruction set for 32-bit code. |
| **BYTE PTR** | Assembler size-override operator telling MASM the operand is exactly 1 byte. |
| **Canvas 2D API** | HTML5 browser API for drawing 2D graphics procedurally using JavaScript. |
| **COAL** | Computer Organization and Assembly Language — the course for which this project was developed. |
| **CORS** | Cross-Origin Resource Sharing — a browser security mechanism that requires servers to opt in to cross-origin requests via HTTP headers. |
| **COFF** | Common Object File Format — the binary format used by Windows .obj and .exe files. |
| **ESI** | Extended Source Index — a general-purpose 32-bit register used here as a pointer register. |
| **extern "C"** | C++ linkage specifier that disables name mangling so a symbol retains its plain C name for the linker. |
| **fetch()** | JavaScript browser API for making asynchronous HTTP requests. |
| **GetLocalTime** | Win32 API function (kernel32.dll) that fills a SYSTEMTIME structure with the current local date and time. |
| **JSON** | JavaScript Object Notation — a lightweight text format for structured data interchange. |
| **MASM** | Microsoft Macro Assembler — the x86 assembler used in this project. |
| **MOVZX** | Move with Zero Extension — copies a smaller operand into a larger register, filling upper bits with zeros. |
| **PROTO** | MASM directive declaring a procedure prototype for use with the INVOKE macro. |
| **Seven-Segment** | A display format using seven line segments (a–g) to render decimal digits, mimicking hardware LED displays. |
| **STDCALL** | A calling convention where the callee cleans the stack. Used by Win32 API and this project's exports. |
| **SYSTEMTIME** | A 16-byte Windows structure holding year, month, day-of-week, day, hour, minute, second, and milliseconds. |
| **Winsock2** | The Windows implementation of the Berkeley sockets API, used here for the HTTP server (ws2_32.dll). |